AgentBudget: Open-source task-level cost, latency, and reliability control for AI agents.

In this i think we should introduce our own harness becoz nowadays in industry the agents are failing becoz of not having good harness

Yes—include a harness, but make it **budget-native**, not a generic "another LangChain." That is the right differentiation: most agent reliability comes from the surrounding harness—execution loop, context assembly, memory, guardrails, retries, stop conditions, and human escalation—not from the model alone. [1] [2]

Updated positioning

> **AgentBudget: An open-source, budget-native agent harness and control plane for task-level cost, latency, and reliability.**

The key phrase is **budget-native harness**. Existing agent frameworks usually add cost tracking afterward; AgentBudget makes the budget a first-class part of the agent loop.

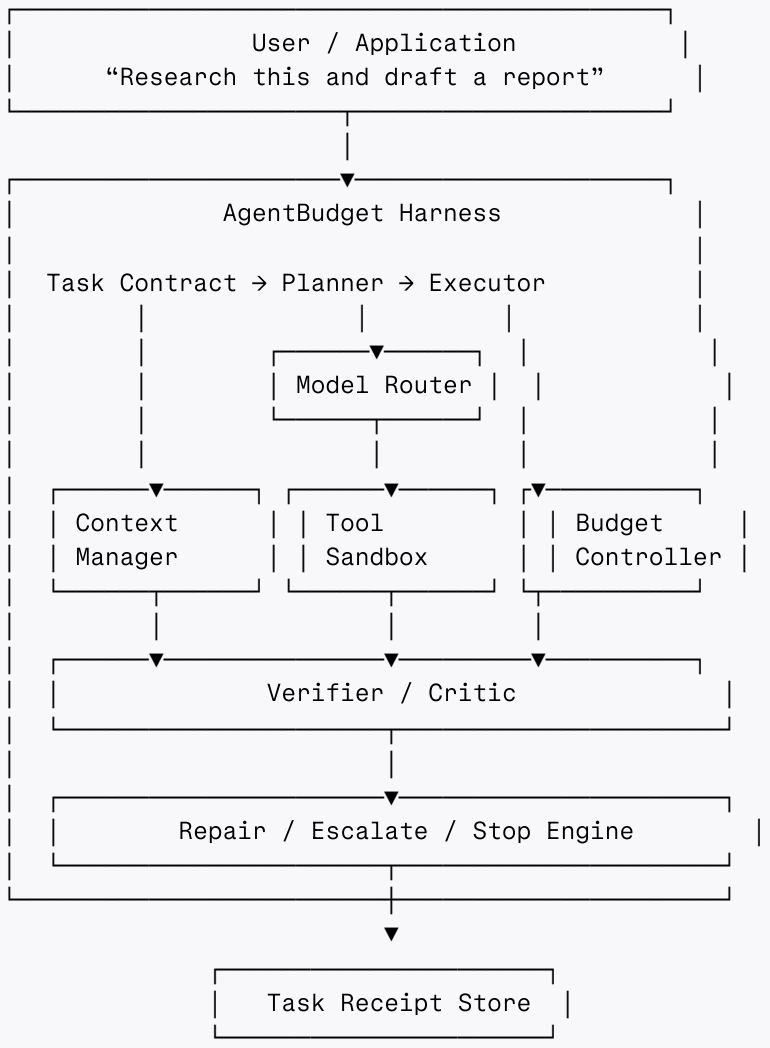
Why a harness matters

Agents commonly fail because of harness-level issues:

- Infinite or near-duplicate retry loops
- Repeated tool calls with the same arguments
- Malformed tool arguments
- Missing stop conditions
- Oversized or irrelevant context
- Unsafe retries on non-idempotent actions
- No persisted state across steps or handoffs
- No clear point to escalate to a human

These are orchestration problems, not necessarily model-intelligence problems. Tool-call failures, retry behavior, timeout policy, permissions, and recovery logic belong in the harness. [3] [4] [5]

Architecture



Core components

1. Task Contract

Every agent task begins with an explicit contract:

```
task:
  goal: "Summarize this repository and identify duplicated logic"
  success_criteria:
    - "Identifies duplicated functions"
    - "Proposes a private helper"
    - "Runs pytest"
  budget:
    max_llm_calls: 12
    max_tokens: 60000
    max_tool_calls: 15
    max_cost_usd: 0.50
    max_wall_time_seconds: 300
  escalation:
```

```
    on_budget_exceeded: "pause_and_ask_human"
    on_repeated_failure: "escalate"
```

This prevents the agent from silently burning tokens with no definition of done.

2. Budget Controller

The controller is enforced in **code**, not merely requested in the system prompt. Production-readiness guidance specifically recommends hard max-step and max-cost limits in code rather than relying on prompt instructions. [2]

It tracks:

- Tokens per model call
- Cost per model call
- Tool-call count
- Retry count
- Wall-clock time
- Estimated remaining budget
- Whether the task is still likely to succeed

3. Context Manager

This is where AgentBudget can save the most money.

It should:

- Deduplicate repeated tool outputs
- Summarize long tool results
- Keep only task-relevant retrieved chunks
- Compress old conversation turns
- Preserve essential state in a structured scratchpad
- Detect context bloat before it reaches the model

Content bloat and retries directly increase inference cost, while KV-cache memory grows with sequence length and affects GPU utilization and batch capacity. [6] [7]

4. Tool Governor

Do not let the model directly execute arbitrary tools.

The harness should validate:

- Is this tool allowed for this task?
- Are arguments schema-valid?

- Is this a read or a write?
- Is retrying safe?
- Has this exact call already failed?
- Should the result be cached?

Retry logic must distinguish safe read retries from dangerous non-idempotent writes, such as payments or deployments. [3]

5. Verifier / Critic

After important steps, check whether the agent is actually progressing:

- Did the code compile?
- Did tests pass?
- Does extracted data match the schema?
- Did the search return new information?
- Is the agent repeating itself?
- Is the partial answer useful?

This creates a feedback loop instead of blindly continuing until the token budget disappears.

6. Repair / Escalation Engine

When something fails, choose deliberately:

Situation	Action
Transient tool timeout	Retry with backoff
Same tool failed twice with same input	Change approach or escalate
Context too large	Compress or summarize
Budget nearly exhausted	Produce best partial answer
Unsafe write operation	Pause for human approval
Repeated model failure	Switch model or escalate

This is the difference between an agent that loops until it fails and one that recovers predictably. [5] [3]

7. Task Receipt

Every run produces a receipt:

```
{
  "task_id": "abc123",
  "status": "completed",
  "success": true,
  "tokens": 41200,
```

```
  "cost_usd": 0.31,
  "llm_calls": 9,
  "tool_calls": 6,
  "retries": 1,
  "wall_time_seconds": 94,
  "waste_detected": [
    "duplicate_web_search",
    "oversized_file_read"
  ],
  "slo_met": true
}
```

This is your strongest demo artifact: it shows exactly where money was wasted and how AgentBudget prevented it.

What makes it different

Existing category	Typical focus	AgentBudget's focus
Agent frameworks	Make it easy to build an agent	Make an agent **cheap, bounded, and reliable**
Observability tools	Show what happened	Prevent waste and enforce limits during the run
Model routers	Choose a model	Control the whole task execution path
Prompt tools	Improve one prompt	Manage context across an entire agent run
Evals	Score final output	Verify progress and stop bad runs early

Agent observability platforms already trace tokens, tool calls, cost, latency, and errors; AgentBudget's added value is **closed-loop enforcement**, not just dashboards. [8] [9] [10]

MVP: build this first

Do not start with multi-agent orchestration. Build a single-agent harness with:

1. Task contract
2. Tool registry with schemas
3. Budget controller
4. Tool-call deduplication
5. Retry policy with safe/unsafe distinction
6. Context summarizer
7. Stop conditions
8. Task receipt
9. Simple web dashboard

A compelling first demo:

```
Task: "Find duplicated pricing logic and make tests pass."
```

```
Without AgentBudget:
- 22 LLM calls
- 18 tool calls
- 4 duplicate file reads
- 3 repeated searches
- 168,000 tokens
- $1.24

With AgentBudget:
- 9 LLM calls
- 6 tool calls
- 0 duplicate reads
- 1 retry
- 43,000 tokens
- $0.29
- Tests passed
```

Important caution

Do **not** claim AgentBudget replaces LangGraph, CrewAI, AutoGen, or OpenAI Agents SDK. Instead, make it interoperable:

- Use AgentBudget as a standalone harness, **or**
- Embed it as a middleware/control plane inside existing frameworks.

That gives you two adoption paths and avoids competing head-on with mature frameworks. Your unique claim should be:

> **"AgentBudget is the first open-source agent harness where task budget, reliability policy, and cost control are enforced as core runtime primitives—not added afterward as observability."**

That is focused, credible, and solves a real production problem. [1] [2] [3]

⁂

1. https://www.fiddler.ai/blog/building-reliable-agents-takes-more-than-better-models
2. https://futureagi.com/blog/agent-harness-production-readiness/
3. https://www.mongodb.com/company/blog/technical/agent-orchestration-tool-use-machinery-underneath
4. https://atlan.com/know/agent-harness-failures-anti-patterns/
5. https://www.openlayer.com/blog/ai-agent-failure-modes-tool-calling-loops-propagation
6. https://www.techtarget.com/it-strategy/feature/The-hidden-costs-of-AI-What-leaders-must-budget
7. https://arxiv.org/html/2504.19720v1
8. https://langwatch.ai/blog/what-is-agent-observability
9. https://www.getorchestra.io/guides/agent-observability
10. https://openobserve.ai/llm-observability/
11. https://atlan.com/know/data-quality-ai-agent-harnesses/
12. https://laminar.sh/article/best-ai-agent-cost-tracking-tools

13. https://github.com/Klepsiphron/agenttrace/

14. https://github.com/MLOps-Courses/agentops-open-course/blob/main/docs/7. Observability/7.3. Costs.md

15. https://github.com/rizzdev/awesome-llm-observability/blob/main/README.md

16. https://github.com/rizzdev/awesome-llm-observability

17. https://www.nops.io/blog/ai-cost-observability/